UNIVERSIDAD TECNOLÓGICA DEL PERÚ (UTP)
Facultad de Ingeniería • Carrera de Ingeniería de Sistemas e Informática
Curso: Integrador I: Sistemas Software (Sección 20065) • Entregable Académico Oficial S09

DISEÑO Y APLICACIÓN DEL PATRÓN MVC EN EL PROYECTO INTEGRADOR

Semana 09 • Arquitectura Modelo-Vista-Controlador, Flujo de Datos End-to-End y Capas SOLID

Estudiante: Ronny Fernando Mendoza Paucar	**Código UTP:** U25302314
Proyecto Integrador: Sistema Inteligente de Inventario y Predicción (SIIP)	**Contexto de Aplicación:** Farmacias Peruanas (InkaFarma / Mifarma)
Repositorio GitHub Oficial: https://github.com/Fern-411/Farmacias	**Modalidad de Entrega:** Individual (Formato PDF)

1. Introducción y Fundamentos del Patrón MVC en el Contexto Farmacéutico

El patrón arquitectónico **Modelo-Vista-Controlador (MVC)** constituye la base de estructuración del **Sistema Inteligente de Inventario y Predicción (SIIP)** para **Farmacias Peruanas (InkaFarma y Mifarma)**. En el desarrollo de software para el retail farmacéutico nacional, la separación estricta de responsabilidades entre la capa de presentación, la lógica de negocio y la persistencia de datos es un **requisito crítico de calidad y auditoría**. Esta separación garantiza el cumplimiento de normativas de salud como las **Buenas Prácticas de Almacenamiento (BPA R.M. 833-2015 DIGEMID)** y los estándares tributarios de **Guías de Remisión Electrónica de SUNAT**.

El objetivo del diseño MVC en el proyecto SIIP es desacoplar completamente la experiencia de usuario (operada por químicos farmacéuticos y administradores de sede) de los motores de persistencia en **PostgreSQL 16** y de los algoritmos de predicción de quiebres, facilitando la mantenibilidad, escalabilidad en microservicios y verificabilidad mediante pruebas unitarias TDD.

2. Descomposición de las Tres Capas del MVC en el Sistema SIIP

La arquitectura del proyecto implementa una variante moderna del patrón MVC adaptada a microservicios desacoplados (API REST + Single Page Application):

Capa MVC	Tecnología y Componentes	Responsabilidad Concreta en el Negocio Farmacéutico
MODELO (Model)	Spring Data JPA, Hibernate, PostgreSQL 16. *Clases:* Usuario, Rol, Medicamento, Inventario, LoteCaducidad, Traslado, AuditoriaEvento.	Encapsula el estado y las reglas de negocio del inventario. Administra las restricciones de integridad relacional (8 tablas normalizadas), el cálculo de stock disponible, estados FEFO y la persistencia inmutable en base de datos.
VISTA (View)	React 18, Vite, Tailwind CSS, Lucide. *Módulos:* LoginPage, DashboardPage, InventorySearchPage, ProductManagementPage, TransfersPage.	Capa visual que interactúa con el usuario final. Presenta la semaforización de stock (Óptimo, Alerta, Quiebre), minimapas de geolocalización de sedes, fotos de medicamentos desde Cloudflare R2 y modales de emisión de guías SUNAT.
CONTROLADOR (Controller)	Spring Boot REST Controllers, Spring MVC. *Clases:* AuthController, ReporteExcelController, InventarioController.	Punto de entrada de las peticiones HTTP (JSON). Sanitiza entradas con Apache Commons, aplica control de tasa (Guava), valida tokens JWT, invoca los servicios de negocio y retorna respuestas con códigos de estado HTTP estándar (200, 201, 401, 423).

3. Procedimientos Metodológicos para la Estructuración del MVC de Inicio a Fin

Para concebir y estructurar el MVC en el proyecto integrador, se ejecutó un procedimiento analítico en **seis fases secuenciales**, garantizando la trazabilidad entre el problema real de la cadena farmacéutica y las líneas de código:

Fase / Procedimiento	Aspectos Analizados	Resultado / Artefacto Técnico Producido
Paso 1: Análisis de Actores y Roles (RBAC)	Determinación de los perfiles operacionales del negocio farmacéutico según normativas DIGEMID y SUNAT.	Matriz de **5 Roles Indispensables:** QF Director Técnico, Admin CD Huachipa, Supervisor Operaciones, Técnico Ventanilla y Auditor DIGEMID.
Paso 2: Descomposición de Módulos Core	Análisis funcional de las necesidades operativas de la cadena: login, inventario geocercano, vademécum, alertas y traslados.	Definición de **5 Módulos de Sistema:** Seguridad (Auth), Inventario Nacional (Haversine), Catálogo Kardex, Predictor FEFO y Despacho GRE.
Paso 3: Diseño de Contratos de Interfaz (DTOs)	Estructuración de las cargas útiles de información (payloads JSON) que viajan entre la Vista y el Controlador.	Especificación de DTOs: LoginRequest, AuthResponse, TransferPayload, StockAdjustmentForm con validación de tipos.
Paso 4: Modelado Conceptual y DAO (Modelo)	Diseño de entidades persistentes, relaciones de cardinalidad, llaves foráneas e índices de rendimiento en base de datos.	Esquema DDL relacional de **8 Tablas en PostgreSQL 16:** sucursales, roles, usuarios, medicamentos, etc.
Paso 5: Implementación de Lógica en Servicios	Aislamiento de las reglas de cálculo (fórmula Haversine, semáforo FEFO, bloqueo a 5 intentos con Guava, reportes POI).	Capa de Servicio orientada a interfaces: IAuthService, IExcelReportService bajo principios SOLID.
Paso 6: Enrutamiento y Exposición de Controladores	Mapeo de endpoints RESTful, sanitización de inputs y vinculación reactiva con los componentes visuales de la interfaz.	Controladores Spring Boot: AuthController.java, ReporteExcelController.java y vistas React desacopladas.

Mapeo Concreto de Clases y Archivos del Proyecto en el Patrón MVC

A continuación se presenta el mapa de archivos reales que integran cada una de las capas del MVC en el repositorio:

Componente MVC	Archivo en el Repositorio GitHub	Función Específica en la Arquitectura
Modelo (Entity)	backend/service-auth/.../model/Usuario.java	Entidad JPA con mapeo a tabla usuarios, hash BCrypt y estado de bloqueo.
Modelo (DAO)	backend/service-auth/.../dao/UsuarioDao.java	Interfaz Spring Data JPA que implementa consultas SQL nativas e indexadas.
Servicio (Service)	backend/service-auth/.../service/impl/AuthServiceImpl.java	Lógica de bloqueo Guava (5 intentos) y auditoría en Logback.
Controlador (REST)	backend/service-auth/.../controllers/AuthController.java	Exposición de endpoints /login, /roles, /usuarios y /health.
Vista (Component)	frontend/src/pages/LoginPage.jsx	Formulario interactivo con honeypot, consentimientos Ley 29733 y selector RBAC.
Vista (Modal)	frontend/src/components/TransferRequestModal.jsx	Modal de cálculo antes ➔ después, selector de cantidad y despacho express.

4. Flujo de la Información de Inicio a Fin en los Casos de Uso Clave

Para responder con rigurosidad técnica a la pregunta: *¿De dónde inicia la información, hacia dónde va y hacia dónde termina?*, se detallan a continuación los tres flujos neurálgicos del sistema SIIP:

Flujo 1: Autenticación Segura y Control de Intentos con Google Guava (CU01)

1. Origen: El usuario (ej. Químico Farmacéutico) introduce sus credenciales en LoginPage.jsx (Vista).
2. Tránsito: Se valida la política de cookies y privacidad (Ley 29733), se verifica el campo honeypot contra bots y se envía un POST /api/v1/auth/login.
3. Control: AuthController (Controlador) sanitiza los datos y delega la operación a AuthServiceImpl (Servicio).
4. Lógica y Persistencia: El servicio consulta el caché en memoria de **Google Guava** (si tiene ≥5 intentos fallidos, arroja error 423 Locked). Si no está bloqueado, consulta a UsuarioDao (DAO/Modelo) que ejecuta una búsqueda en PostgreSQL. Se valida la contraseña con BCryptPasswordEncoder.
5. Auditoría y Destino: Se registra el evento en auditoria_eventos (PostgreSQL) y en siip-security-audit.log (Logback). El Controlador retorna el token JWT y los datos de sede; la Vista renderiza el panel según el rol RBAC.

Flujo 2: Consulta de Inventario Geocercano y Algoritmo Haversine (CU02)

1. Origen: Un paciente presenta una receta en ventanilla en InkaFarma San Isidro requiriendo un antibiótico en quiebre (Amoxidal 500 mg). El QF busca el producto en InventorySearchPage.jsx (Vista).
2. Tránsito: La interfaz emite una solicitud con las coordenadas geográficas (latitud, longitud) de la sede actual y el ID del medicamento.
3. Lógica Algorítmica: El Servicio calcula la distancia geodésica exacta en kilómetros entre la sede del paciente y las otras 14 sedes de la red nacional mediante la **Fórmula Haversine**:

$$d = 2r \cdot \arcsin\left(\sqrt{\sin^2(\Delta lat/2) + \cos(lat1)\cdot\cos(lat2)\cdot\sin^2(\Delta lng/2)}\right)$$

4. Consulta del Modelo: Se cruzan las distancias con las existencias disponibles en la tabla inventarios (Modelo), filtrando solo sedes con stock mayor a cero en un radio óptimo (<10 km).
5. Destino: La información se retorna ordenada ascendentemente por distancia. La Vista despliega las sedes más cercanas (ej. Mifarma Miraflores a 2.3 km con 95 cajas) y habilita el minimapa interactivo.

Flujo 3: Traslado Express Inter-Sedes con Emisión de Guía GRE SUNAT (CU03)

1. Origen: El QF selecciona la sede donante e indica la cantidad de cajas (ej. 20 cajas) en TransferRequestModal.jsx (Vista).
2. Tránsito: Se genera un payload transaccional que incluye ID de origen, destino, medicamento, cantidad y justificación técnica.
3. Control y Negocio: El Controlador recibe la petición y el Servicio ejecuta una **transacción atómica ACID**:
- *Débito:* Disminuye el stock en la sede donante (Miraflores: 95 → 75 cajas).
- *Crédito:* Incrementa el stock en la sede receptora (San Isidro: 4 → 24 cajas).
- *Kardex:* Registra dos asientos formales (SALIDA_TRASLADO y ENTRADA_TRASLADO).
- *Documentos Regulatorios:* Genera la Guía de Remisión Electrónica SUNAT con código QR/HASH y el Acta de Recepción Conforme DIGEMID BPA.

4. Persistencia: Los cambios se graban en las tablas inventarios y traslados de PostgreSQL y se sincronizan en el almacenamiento reactivo.
5. Destino: La Vista actualiza en caliente el contador de stock a **24 cajas**, persistiendo permanentemente incluso tras recargar la página (F5).

Paso del Flujo	Capa MVC Activa	Entrada (Input)	Salida / Transformación (Output)
1. Entrada	VISTA (React)	Interacción usuario (Clic, Input)	Payload JSON estructurado
2. Despacho	CONTROLADOR (Spring)	HTTP Request + Headers JWT	Validación sintáctica y delegación
3. Cómputo	SERVICIO (SOLID)	Parámetros de negocio limpios	Cálculo Haversine / Transacción ACID
4. Persistencia	MODELO (DAO/JPA)	Entidades en memoria	Sentencias SQL en PostgreSQL 16
5. Respuesta	VISTA (React UI)	HTTP Response (Status 200/201)	Renderizado visual y persistencia F5

5. Integración Tecnológica de Librerías Exigidas dentro del MVC

En estricto cumplimiento con los requerimientos técnicos de la cátedra de Integrador I, se integraron cuatro bibliotecas Java empresariales, articuladas orgánicamente dentro del flujo MVC:

Librería Integrada	Ubicación en el MVC	Propósito Técnico y Caso de Uso en el SIIP
Google Guava (v33.0.0)	Capa de Servicio (AuthService)	Implementación de LoadingCache<String, Integer> para tasa de intentos fallidos. Bloquea la cuenta automáticamente por 15 minutos al 5.° fallo consecutivo antes de saturar PostgreSQL.
Apache POI (v5.2.5)	Capa de Control / Servicio (ExcelReport)	Generación de libros contables y actas de auditoría oficial en formato Excel (.xlsx) con celdas estilizadas corporativamente (#002B66) para DIGEMID y SUNAT.
Apache Commons (v3.14.0)	Capa de Controlador / Util	Sanitización estricta de cadenas (StringUtils) y validación de nulidad para prevenir inyecciones SQL y ataques XSS en formularios de búsqueda y login.
Logback / SLF4J	Transversal a todo el MVC	Bitácora rotativa de auditoría de seguridad en logs/siip-security-audit.log, registrando fecha, hora, usuario, IP y resultado de cada transacción del sistema.

6. Conclusiones Técnicas del Diseño MVC

1. **Alta Cohesión y Bajo Acoplamiento:** La aplicación del patrón MVC permitió que cambios sustanciales en el Modelo (como la migración a 8 tablas en PostgreSQL 16 con hash BCrypt) no afectaran en absoluto los componentes visuales de la Vista ni los contratos del Controlador.

2. **Resiliencia y Persistencia Garantizada:** El diseño del flujo de información resolvió el problema de volatilidad en traslados inter-sucursales. Las transferencias de medicamentos (como Amoxidal 500 mg) descuentan de origen y acreditan a destino en una sola operación atómica respaldada en base de datos, manteniéndose inalterables ante recargas de página (F5).

3. **Auditoría y Trazabilidad DIGEMID/SUNAT:** Cada componente del MVC contribuye a la cadena de custodia del medicamento: la Vista captura la intención del farmacéutico, el Controlador valida la legalidad de la solicitud y el Modelo registra la emisión de Guías Electrónicas GRE y Actas BPA.

DECLARACIÓN DE AUTORÍA Y ENTREGA INDIVIDUAL:
El suscrito estudiante declara que el diseño, los diagramas de flujo, la modelación conceptual y la estructuración del patrón MVC documentados en este informe han sido elaborados de manera individual para el Proyecto Integrador SIIP (Farmacias Peruanas), cumpliendo con los estándares de ingeniería de software evaluados en la Semana 09 del curso.

Firma del Estudiante: _____
Ronny Fernando Mendoza Paucar | *Código UTP:* ***U25302314***
Carrera de Ingeniería de Sistemas e Informática — Universidad Tecnológica del Perú
Lima, Octubre de 2026

ENLACE DIRECTO AL REPOSITORIO GITHUB OFICIAL DEL PROYECTO:
https://github.com/Fern-411/Farmacias
Estructura completa de paquetes MVC: /backend/service-auth/ (Model, DAO, Controller) y /frontend/src/ (Views).